Restaurant Management System — Full Explanation of src/main/java/com/restaurant

1. Project Overview

This project is a Java-based restaurant management application. The folder `src/main/java/com/restaurant` contains the main logic of the system. It is divided into packages such as `config`, `model`, `service`, `controller`, `repository`, `util`, `data`, `enums`, and `exception`.

The purpose of the project is to manage:

- customers
- orders
- billing
- kitchen tasks
- stock and inventory
- reservations
- users and roles
- payments
- reports

The code follows layered architecture:

- Model = data and objects
- Repository = database access logic
- Service = business rules
- Controller = UI interaction
- Util = helper tools
- Config = connection settings

2. Main Class

`Main.java`

This is the entry point of the project. It starts the JavaFX application and loads the login screen.

Important ideas:

- `extends Application` means the class is a JavaFX app.
- `start(Stage stage)` creates the window and loads `login.fxml`.
- `loadLoginScreen()` loads the login screen from the FXML file.
- `init()` runs before the UI is shown and starts background services.
- `stop()` stops background tasks when the app closes.

- A global exception handler is installed so crashes are logged instead of silently failing.
- Background threads handle kitchen processing and database health monitoring.

This file shows multi-threading, JavaFX navigation, and exception handling.

3. Config Package

`AppConfig.java`

This class reads application settings from `application.properties`.

It stores important values such as:

- database URL
- username
- password
- app name
- currency symbol
- tax rates

The class uses a `Properties` object, which is a Java data structure for storing key-value pairs. It loads the file once and gives methods to read values like `getString()`, `getInt()`, and `getDouble()`.

`DatabaseConfig.java`

This class connects the Java application to the MySQL database.

It contains:

- URL, username, password, and driver name
- a singleton instance (`getInstance()`)
- `Class.forName(driver)` to register the MySQL JDBC driver
- `getConnection()` to open a connection
- `testConnection()` to verify database availability

This class is important because every repository uses database connection through it.

4. Model Package

The model package contains the real-world entities of the system. These Java classes represent objects used throughout the application.

`User.java`

This is the base class for all system users. It contains common attributes like:

- ID
- full name
- username
- email
- phone
- password hash
- user role

It demonstrates abstraction because all users share common features.

`Employee.java`

This class extends `User` and represents staff members such as managers, waiters, cashiers, and kitchen staff.

`Admin.java`

Represents the system administrator.

`Manager.java`

Represents the manager role. This user can manage menu, inventory, staff, and reservations.

`Waiter.java`

Represents a waiter who takes orders and handles table service.

`Cashier.java`

Represents a cashier who handles payment and billing.

`KitchenStaff.java`

Represents kitchen staff who prepare food and update order status.

`Customer.java`

This class represents restaurant customers.

It stores customer details like name, phone number, email, and billing information.

`MenuItem.java`

This class represents a food item in the restaurant menu.

Typical fields include:

- item name

- category
- price
- preparation time
- availability

`MenuCategory.java`

This class groups menu items into categories like starters, mains, beverages, desserts.

`MenuItemIngredient.java`

This class links a menu item to its ingredients and quantities. It is useful for recipe and inventory planning.

`InventoryItem.java`

This is a stock item such as rice, flour, tomato, chicken, etc.

It stores:

- item name
- quantity
- unit
- minimum stock level

`Order.java`

This class represents a customer order.

It stores:

- order ID
- customer details
- table number
- order status
- list of items
- subtotal
- tax
- service charge
- total amount

`OrderItem.java`

This class represents one item inside an order. It includes the menu item, quantity, and price.

`RestaurantTable.java`

This class represents a table in the restaurant. It tracks table number, capacity, and status.

`Reservation.java`

This class stores reservation information such as guest name, phone, date, time, table, and status.

`Payment.java`

This is the general payment class. It can be used for different payment methods.

`CashPayment.java`

This class extends `Payment` and stores cash payment details.

`CardPayment.java`

This class extends `Payment` and stores card payment details.

`UPIPayment.java`

This class extends `Payment` and stores UPI payment details.

`Supplier.java`

This class stores supplier company details such as name, contact, and goods supplied.

`Purchase.java`

This class represents a purchase order or stock purchase from suppliers.

`PurchaseItem.java`

This class represents one item inside a purchase record.

5. Enum Package

The `enums` package contains special classes with fixed values.

`UserRole.java`

This enum defines user roles:

- ADMIN
- MANAGER
- WAITER
- CASHIER
- KITCHEN_STAFF

This prevents invalid role values.

`TableStatus.java`

Values may include:

- AVAILABLE
- OCCUPIED
- RESERVED
- CLEANING

`OrderStatus.java`

Values may include:

- CONFIRMED
- PREPARING
- READY
- SERVED
- CANCELLED

`PaymentMethod.java`

Defines possible payment types such as CASH, CARD, and UPI.

`PaymentStatus.java`

Represents whether payment is PENDING, SUCCESSFUL, or FAILED.

`ReservationStatus.java`

Defines whether a reservation is ACTIVE, CANCELLED, or COMPLETED.

These enums improve safety and readability by limiting invalid values.

6. Repository Package

Repository classes are responsible for database access. They connect the application to MySQL.

`UserRepository.java`

Handles all user-related database operations:

- login check
- create user
- update user
- find by username

- role validation

`CustomerRepository.java`

Handles customer data operations.

`MenuRepository.java`

Handles menu items from the database.

`InventoryRepository.java`

Handles stock records and inventory updates.

`OrderRepository.java`

Manages orders and order details.

`PaymentRepository.java`

Stores payment data to the database.

`TableRepository.java`

Handles table management in the database.

`ReservationRepository.java`

Handles reservation records.

`SupplierRepository.java`

Stores and retrieves supplier data.

`MenuItemIngredientRepository.java`

Handles ingredient relationships for menu recipes.

Repository classes separate data access logic from business logic and keep the code modular.

7. Service Package

The service package contains business logic. This is where the actual rules of the application are written.

`AuthenticationService.java`

This class handles login, registration, and role checks.

It uses:

- `UserRepository` to find users
- `PasswordUtil` to validate hashed passwords
- `ValidationUtil` to check input values
- `DataStructureManager` to track sessions

This is a good example of business logic being separated from UI code.

`OrderService.java`

This class manages order creation, validation, and status changes.

It ensures that orders are valid before being stored.

`BillingService.java`

This class creates invoice and receipt text. It calculates totals, tax, service charges, and discounts.

`InventoryService.java`

This class manages stock levels and checks whether ingredients are available before placing an order.

`MenuService.java`

This class handles menu operations such as adding, updating, or removing items.

`MenuRecipeService.java`

This class is responsible for recipe logic and ingredient requirements for each menu item.

`TableService.java`

This class manages table availability and handles reservation logic.

`ReservationService.java`

This class checks for table conflicts and validates reservations.

`PaymentService.java`

This class handles payment processing and status updates.

`SupplierService.java`

This class manages supplier records and purchase tracking.

`KitchenService.java`

This class handles kitchen workflow and preparation status.

`KitchenOrderProcessor.java`

This is a background worker that continuously processes kitchen orders and updates their status.

`DatabaseHealthMonitor.java`

This class monitors database connection health and warns if the system is disconnected.

`AsyncReportGenerator.java`

This class generates reports in the background without blocking the UI.

`ReportService.java`

This class prepares summary reports for business analysis.

`CustomerService.java`

This class manages customer details and customer-related operations.

8. Controller Package

Controller classes are connected to the JavaFX UI. They link buttons, text fields, labels, and tables in the FXML screens to Java code.

`LoginController.java`

Handles login form actions.

`DashboardController.java`

Controls the main dashboard screen.

`DashboardHomeController.java`

Manages the home dashboard content and summary cards.

`MenuController.java`

Controls menu display and item operations.

`OrderController.java`

Handles order entry and order management.

`BillingController.java`

Responsible for applying discount, showing total calculations, and previewing the receipt.

`PaymentController.java`

Handles customer payment transactions.

`InventoryController.java`

Handles stock updates and supplier records.

`KitchenController.java`

Controls kitchen order display and preparation tasks.

`ReservationController.java`

Manages reservation forms and booking logic.

`CustomerController.java`

Handles customer registration and management.

`SupplierController.java`

Handles supplier operations.

`TableController.java`

Handles table assignment and table status updates.

`ReportController.java`

Controls report generation and display in the UI.

The controller classes are used to connect user actions to service methods. They are not supposed to contain complex business logic; they mostly coordinate events.

9. Util Package

Utility classes are helper classes used across the application.

`ValidationUtil.java`

This class checks data quality. It validates:

- email format
- phone number
- non-empty strings

- numeric values

`PasswordUtil.java`

This class encrypts and verifies passwords using hashing.

`CurrencyUtil.java`

This class formats money values and currency symbols.

`DateTimeUtil.java`

This class helps with date and time operations.

`AlertUtil.java`

This class displays popup alerts in the JavaFX UI.

`ActivityLogger.java`

This class writes application logs to files for auditing and debugging.

`BillReceiptWriter.java`

This class writes bill or receipt information into a text or file format.

`ReportExporter.java`

This class exports generated reports to files.

10. Data Package

This package contains helper data structures used by the app.

`DataStructureManager.java`

This class manages application session data and runtime structures.

`OrderQueue.java`

This is a queue used to handle orders in sequence, especially in kitchen processing.

`SearchSortUtil.java`

This class helps sort and search records efficiently.

These structures help in organizing large amounts of data in a clean way.

11. Exception Package

These classes represent custom errors and invalid conditions.

Examples:

- `InvalidLoginException` — login values are wrong
- `InvalidOrderException` — order data is invalid
- `InvalidReservationException` — reservation is invalid
- `InsufficientStockException` — not enough inventory
- `MenuItemNotAvailableException` — menu item unavailable
- `TableNotAvailableException` — table is booked or not available
- `PaymentFailedException` — payment operation failed
- `DatabaseOperationException` — database error

Custom exceptions make the app easier to understand and easier to debug.

12. OOP Concepts Used in This Project

This project is a clear example of object-oriented programming.

Encapsulation

The class fields are protected by private variables and accessed through getter/setter methods.

Inheritance

Classes like `Manager`, `Waiter`, and `Cashier` extend `Employee`, which extends `User`.

Polymorphism

Different payment types share the same base type `Payment` but behave differently.

Abstraction

Classes hide implementation details and expose only necessary operations.

Composition

Orders are made of multiple `OrderItem` objects. Restaurants contain tables and customers.

Singleton Pattern

`AppConfig`, `DatabaseConfig`, and some service classes use a singleton pattern so only one instance is created.

Layered Architecture

The system is divided into model, repository, service, and controller layers.

13. Full Working Flow of the Application

A common flow in the project is:

1. User logs in through `LoginController`.
2. `AuthenticationService` verifies username and password.
3. If valid, the dashboard opens.
4. User takes orders from the menu.
5. `OrderService` validates and stores the order.
6. Kitchen staff receives the order and updates status.
7. When served, billing screen calculates subtotal, tax, discount, and total.
8. `BillingService` creates a receipt.
9. `PaymentService` handles payment completion.
10. Data is saved in the database through repository classes.

This flow is a perfect academic example of how an enterprise software system works.

14. Conclusion

The main folder contains a complete restaurant management system built using Java and JavaFX. Every package has a specific role.

- `model` stores the data objects
- `repository` accesses the database
- `service` handles the business logic
- `controller` connects the UI
- `config` handles database settings
- `util` provides support features
- `exception` handles errors
- `enums` keeps valid values fixed

This structure is easy to explain in a viva or class presentation because each package represents a real-world business layer.